

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 «Компьютерные науки и прикладная математика»

Кафедра 804 «Теория вероятностей и компьютерное моделирование»

Курсовая работа на тему:
«Метод наименьших квадратов»
Вариант №16

Выполнила студентка группы: М8О-305Б-22

Хакимова Александра Эльдаровна

Преподаватель: Игнатов Алексей Николаевич

Оценка: _____

Дата: _____

Москва, 2024

Содержание

1. Постановка задачи	3
1.1. Описание модели	3
1.2. Моделирование данных	3
1.3. Задание	3
2. Выполнение работы	4
Случай 1	4
Задание 2.1. Вычисление порядка многочлена для модели и оценка параметров.	5
Задание 2.2. Доверительные интервалы для параметров.	7
Задание 2.3. Доверительные интервалы для полезного сигнала.	8
Задание 2.4. Графическое представление.	9
Задание 2.5. Гистограмма распределения случайной ошибки.	13
Задание 2.6. Оценка дисперсии случайной ошибки.	15
Задание 2.7. Проверка гипотезы о том, что закон распределения ошибки наблюдения является нормальным.	16
Случай 2	17
Задание 2.1. Вычисление порядка многочлена для модели, нахождение оценки параметров.	18
Задание 2.2. Доверительные интервалы для параметров.	19
Задание 2.3. Доверительные интервалы для полезного сигнала.	20
Задание 2.4. Графическое представление.	21
Задание 2.5. Гистограмма распределения случайной ошибки.	25
Задание 2.6. Оценка дисперсии случайной ошибки.	27
Задание 2.7. Проверка гипотезы о том, что закон распределения ошибки наблюдения является нормальным.	28
Приложения	29
Приложение 1	29
Приложение 2	30
Приложение 3	31
Приложение 4	32
Приложение 5	32
Приложение 6	33

1. Постановка задачи

1.1. Описание модели

Модель полезного сигнала имеет вид:

$$\varphi(x) = \theta_0 + \theta_1 x + \dots + \theta_m x^m. \quad (1)$$

Рассматривается модель наблюдений

$$y_k = \theta_0 + \theta_1 x_k + \dots + \theta_m x_k^m + \varepsilon_k, \quad k = \overline{1, n}, \quad (2)$$

где $\varepsilon_1, \dots, \varepsilon_n$ – независимые центрированные и одинаково распределённые случайные величины.

1.2. Моделирование данных

Смоделировать два набора наблюдений на основе модели (2) для следующих случаев:

1 случай	2 случай
$m = 3, \varepsilon_k \sim N(0, \sigma^2)$	$m = 2, \varepsilon_k \sim R(-\sqrt{3}\sigma, \sqrt{3}\sigma)$
$x_k = -4 + k \cdot \frac{8}{n}, \quad k = \overline{1, n}, \quad n = 40.$	

Параметры задания определяются номером варианта (см. ниже).

Вариант №16
$\theta_0 = -8, \theta_1 = 4, \theta_2 = 6, \theta_3 = -0.11, \sigma^2 = 2.0$

1.3. Задание

Для обоих случаев выполнить по очереди следующие задания.

1. Подобрать порядок многочлена $\hat{m} \geq 1$ в модели (1), используя критерий Фишера на уровне значимости 0.05, и вычислить оценки неизвестных параметров $(\theta_0, \dots, \hat{\theta}_m)$ методом наименьших квадратов.
2. В предположении нормальности ошибок построить доверительные интервалы уровней надёжности $\alpha_1 = 0.95$ и $\alpha_1 = 0.99$ для параметров $(\theta_0, \dots, \hat{\theta}_m)$.
3. В предположении нормальности ошибок построить доверительные интервалы уровней надёжности $\alpha_1 = 0.95$ и $\alpha_1 = 0.99$ для полезного сигнала (1).
4. Представить графически
 - истинный полезный сигнал,
 - набор наблюдений,
 - оценку полезного сигнала, полученную в шаге 1,
 - доверительные интервалы полезного сигнала, полученные в шаге 3.
5. По остаткам регрессии построить оценку плотности распределения случайной ошибки наблюдения в виде гистограммы.
6. В предположении нормальности ошибок вычислить оценку максимального правдоподобия дисперсии σ^2 случайной ошибки.
7. По остаткам регрессии с помощью χ^2 -критерия Пирсона на уровне значимости 0.05 проверить гипотезу о том, что закон распределения ошибки наблюдения является нормальным.

2. Выполнение работы

Случай 1

В первом случае вектор ошибок распределён нормально $m = 3$, $\varepsilon_k \sim \mathcal{N}(0, \sigma^2)$.
Сгенерируем вектор ошибок:

Код:

```
import numpy as np
import scipy
from scipy import stats
import math
import sympy as sp
from scipy.stats import t
import matplotlib.pyplot as plt
from sympy.abc import x

# Определение начальных параметров
N=16
param_0 = -8
param_1 = 4
param_2 = 6
param_3 = -0.11
total_points = 40
variance = 2 ** 0.5
degree = 3

# Список параметров
params = [param_0, param_1, param_2, param_3]

np.random.seed(522) # Устанавливаем начальное значение генератора случайных чисел
error_terms = np.random.normal(0, variance, total_points) # Генерируем случайные
ошибки

x_points = np.array([-4 + k * (8 / total_points) for k in range(1, total_points+1)])

# Вычисление функции phi
phi_function = sum([params[i] * (x_points ** i) for i in range(degree + 1)])
# Генерация y_k с учетом шума
y_values = phi_function + error_terms
error_terms
```

0.54839945, -0.32789123, 0.54052891, -3.32249816, 0.6472887 ,
 -3.30319459, 1.45674527, 1.29281645, 0.0712353 , -1.15413148,
 -1.61245658, -0.65240255, 0.48210922, -2.39969596, 2.27794007,
 -2.29040565, 1.80882755, 0.26581746, 0.26803517, -0.98841832,
 -0.42120178, 0.57488936, 0.91795948, 3.09597981, -1.40122118,
 0.83343362, -0.0653488 , 1.09836816, 0.6515438 , 1.27220107,
 0.76303051, -0.12092966, 1.10908195, -1.26706525, -0.72247556,
 -1.25402262, -0.21892222, -0.92820107, -0.17575224, -2.09536272

Задание 2.1 Вычисление порядка многочлена для модели, нахождение оценки параметров.

Я намерена оценить параметры θ с использованием метода наименьших квадратов, чтобы получить в результате вектор $\hat{\theta}$.

С помощью критерия Фишера мне нужно определить порядок модели $Y=A\theta+\varepsilon$. Сформулирую основную гипотезу в общем матричном виде $H_0: B\hat{\theta}_m = C$, предполагая, что порядок модели m равен значению, которое нужно проверить. Альтернативная гипотеза подразумевает, что $H_1: B\hat{\theta}_m \neq C$.

Для проверки основной гипотезы следует рассчитать статистику Фишера, которая вычисляется по следующей формуле:

$$F(Z_n) = \frac{(B\hat{\theta}-C)^T (B(A^T A)^{-1} B^T)^{-1} (B\hat{\theta}-C)}{\frac{q}{n-(p+1)} (Y-A\hat{\theta})^T (Y-A\hat{\theta})}$$

где n — это количество наблюдений, p — число оцениваемых параметров $p=m+1$, а q — количество строк в матрице B .

Для проверки критерия требуется рассчитать Z и рассмотреть – попала ли

статистика в критическую область, которая имеет вид: $G = (f_{1-\alpha, q, (n-(p+1))}^{+}, \infty)$, где $f_{1-\alpha, q, (n-(p+1))}^{+}$ -квантиль распределения Фишера с уровнем надежности $1 - \alpha$ и параметрами $q, n - (p + 1)$.

Если статистика попала в этот промежуток – принимается альтернативная гипотеза. Иначе – основная.

МНК-оценки параметров $\hat{\theta}$ вычисляются по формуле:

$$\hat{\theta} = (A^T A)^{-1} A^T Y$$

```
def estimate_parameters(degree, transposed_matrix, observed_y):
```

```
    mat_a = transposed_matrix.T
```

```
    cov_matrix = np.linalg.inv(np.dot(transposed_matrix, mat_a))
```

```
    temp_vector = np.dot(transposed_matrix, observed_y)
```

```
    temp_vector = temp_vector.T
```

```
    estimated_params = np.dot(cov_matrix, temp_vector)
```

```
    variance_jj = cov_matrix[degree, degree]
```

```
    errors = observed_y - np.dot(mat_a, estimated_params)
```

```
    sum_squared_errors = np.dot(errors.T, errors)
```

```
    Z_statistic = (total_points - (degree + 1)) * (estimated_params[degree] ** 2) /  
(variance_jj * sum_squared_errors)
```

```
    return Z_statistic, estimated_params, sum_squared_errors, cov_matrix
```

```
for current_degree in range(1, 5):
```

```
    if current_degree == 0:
```

```
        transposed_matrix = np.array([np.ones(total_points)])
```

```
    else:
```

```

        transposed_matrix = np.array([np.ones(total_points)] + [x_points**i for i in
range(1, current_degree + 1)])

        Z_stat, estimated_params, residual_sum, cov_matrix =
estimate_parameters(current_degree, transposed_matrix, y_values)

        print(f"Степень {current_degree}: Z = {Z_stat}, Параметры: {estimated_params}")

```

Степень 1: Z = 4.447866249905908, Параметры: [23.72881646 4.16549378]

Степень 2: Z = 7691.243194638434, Параметры: [-7.59330242 2.98797051 5.88761633]

Степень 3: Z = 44.96925521280759, Параметры: [-7.73738175 4.42575464 5.93275397 -0.15045878]

Степень 4: Z = 0.09178725139682122, Параметры: [-7.66299881e+00 4.43513074e+00 5.88601081e+00
-1.51831851e-01 3.43267021e-03]

$f_1 = 4.098$	$f_2 = 4.105$	$f_3 = 4.113$	$f_3 = 4.121$
4.448	7691.24	44.969	0.0918
m=1	m=2	m=3	m=4

При m=4 статистика не попала в критическую область, значит гипотеза

H_0 принимается. Получается, что порядок модели для оценки сигнала будет $\hat{m} = 4-1=3$.

$$\hat{\theta}_{(1 \times 4)} = (-7.73738175 \ 4.42575464 \ 5.93275397 \ -0.15045878)$$

Задание 2.2. Доверительные интервалы для параметров.

Для построения доверительных интервалов уровня надежности $\alpha_1 = 0.95$ и $\alpha_2 = 0.99$ для параметров модели, определенных в предыдущем пункте, будем использовать следующую формулу:

$$P\left(\hat{\theta}_i - t_{1-\frac{\alpha}{2}, n-m-1} \sqrt{\hat{\sigma}^2 c_{ii}} \leq \theta_i \leq \hat{\theta}_i + t_{1-\frac{\alpha}{2}, n-m-1} \sqrt{\hat{\sigma}^2 c_{ii}}\right) = 1 - \alpha,$$

где $\hat{\sigma}^2 = \frac{S(\hat{\theta})}{n-m-1} = \frac{\|E\|^2}{n-m-1}$ – оценка дисперсии ошибок, $t_{1-\frac{\alpha}{2}, n-m-1}$ – экстремальная статистика для уровня надежности $[1 - \alpha_1, 1 - \alpha_2]$, m – порядок модели, вычисленный в прошлом пункте, c_{ii} – элемент на пересечении i столбца и i строки матрицы $(A^T A)^{-1}$.

$$t_{0.975, 36} = 2.0281; t_{0.995, 36} = 2.7195$$

$$S(\hat{\theta}) = (Y - A\hat{\theta})^T (Y - A\hat{\theta}) = 67.275$$

$$\hat{\theta}_0 - t_{0.975, 36} \sqrt{\frac{c_{11} S(\hat{\theta})}{40-4}} = -8.396$$

$$\hat{\theta}_0 + t_{0.975, 36} \sqrt{\frac{c_{11} S(\hat{\theta})}{40-4}} = -7.078$$

$$\hat{\theta}_0 - t_{0.995, 36} \sqrt{\frac{c_{11} S(\hat{\theta})}{40-4}} = -8.620$$

$$\hat{\theta}_0 + t_{0.995, 36} \sqrt{\frac{c_{11} S(\hat{\theta})}{40-4}} = -6.854$$

```
# Определение границ доверительных интервалов
degrees_of_freedom = total_points - degree - 1
confidence_level_1 = 0.95
confidence_level_2 = 0.99
t_critical_1 = stats.t.ppf(0.975, degrees_of_freedom)
```

```

t_critical_2 = stats.t.ppf(0.995, degrees_of_freedom)
print("Критические значения t для уровней доверия:")
print(t_critical_1)
print(t_critical_2)

# Сообщение о доверительных интервалах
print('\nДля уровня надежности alpha1 = 0.95:')

if len(estimated_params) == 1:
    i = 0 # Обращаемся только к первому параметру
    conf_interval_left = estimated_params[i] - t_critical_1 * math.sqrt(cov_matrix[i][i]
* residual_sum / (total_points - degree - 1))
    conf_interval_right = estimated_params[i] + t_critical_1 *
math.sqrt(cov_matrix[i][i] * residual_sum / (total_points - degree - 1))
    print(f"Доверительный интервал для параметра 0: {conf_interval_left} <= theta0 <=
{conf_interval_right}")
else:
    for i in range(len(estimated_params)):
        conf_interval_left = estimated_params[i] - t_critical_1 *
math.sqrt(cov_matrix[i][i] * residual_sum / (total_points - degree - 1))
        conf_interval_right = estimated_params[i] + t_critical_1 *
math.sqrt(cov_matrix[i][i] * residual_sum / (total_points - degree - 1))

        print(f"Доверительный интервал для параметра {i}: {conf_interval_left} <=
theta{i} <= {conf_interval_right}")

print('\nДля уровня надежности alpha2 = 0.99:')

if len(estimated_params) == 1:
    i = 0 # Обращаемся только к первому параметру
    conf_interval_left = estimated_params[i] - t_critical_2 * math.sqrt(cov_matrix[i][i]
* residual_sum / degrees_of_freedom)
    conf_interval_right = estimated_params[i] + t_critical_2 *
math.sqrt(cov_matrix[i][i] * residual_sum / degrees_of_freedom)
    print(f'Для параметра {i}: {conf_interval_left} <= theta{i} <=
{conf_interval_right}')
else:
    print(f'Предупреждение: Параметр {i} выходит за пределы массива оцененных
параметров.')

```

Критические значения t для уровней доверия:

2.0280940009804502

2.719484630449974

Для уровня надежности alpha1 = 0.95:

Доверительный интервал для параметра 0: -8.396301026376598 <= theta0 <= -7.0784624766624
 Доверительный интервал для параметра 1: 3.9509148236030778 <= theta1 <= 4.90059446412609
 Доверительный интервал для параметра 2: 5.83970848667523 <= theta2 <= 6.0257994465835925
 Доверительный интервал для параметра 3: -0.19596259451777576 <= theta3 <= -0.1049549711793073

Для уровня надежности alpha2 = 0.99:

Доверительный интервал для параметра 0: -8.620930955755027 <= theta0 <= -6.853832547283972
 Доверительный интервал для параметра 1: 3.78903879514035 <= theta1 <= 5.062470492588818
 Доверительный интервал для параметра 2: 5.807988668499289 <= theta2 <= 6.0575192647595335
 Доверительный интервал для параметра 3: -0.2114751442211557 <= theta3 <= -0.0894424214759675

Задание 2.3. Доверительные интервалы для полезного сигнала.

Для построения доверительных интервалов уровня надежности $\alpha_1 = 0.95$ и $\alpha_2 = 0.99$ для полезного сигнала, будем использовать следующую формулу:

$$P\left(\sum_{i=0}^m \hat{\theta}_i x^i - t_{1-\frac{\alpha}{2}, n-m-1} \sqrt{X^T (A^T A)^{-1} X \frac{s(\hat{\theta})}{n-m-1}} \leq \sum_{i=0}^m \theta_i x^i \leq \sum_{i=0}^m \hat{\theta}_i x^i + t_{1-\frac{\alpha}{2}, n-m-1} \sqrt{X^T (A^T A)^{-1} X \frac{s(\hat{\theta})}{n-m-1}}\right)$$

где $X = (1 \ x \ : \ x^m)$, $\frac{s(\hat{\theta})}{n-m-1} = \frac{\|E\|^2}{n-m-1}$ – оценка дисперсии ошибок, $t_{1-\frac{\alpha}{2}, n-m-1}$ – экстремальная статистика для уровня надежности $[\alpha = 1 - \alpha_1 \ \alpha = 1 - \alpha_2]$, m – порядок модели, вычисленный в задании 2.1

Уровни надежности для $\alpha_1 = 0.95$:

$$1 - \frac{\alpha_1}{2} = 0.975, \ t_{0.975, 36} = 2.0281;$$

Уровни надежности для $\alpha_1 = 0.99$:

$$1 - \frac{\alpha_2}{2} = 0.995, \ t_{0.995, 36} = 2.7195;$$

Произведем расчеты:

$$\hat{\theta}_0 + \hat{\theta}_1 x + \dots + \hat{\theta}_m x^m - t_{0.975,36} \sqrt{\frac{s(\hat{\theta})}{n-m-1} f^T (A^T A)^{-1} f} = -0.1505x^3 + 5.933x^2 + 4.426x - 2.0281 \sqrt{x^3(0.00050x^3 - 0.00015x^2 - 0.0048x + 0.00048) + 0.00048x^3}$$

$$\sqrt{+x^2(-0.00015x^3 + 0.00210x^2 + 0.00103x - 0.0111) - 0.0111x^2 + x(-0.0048x^3 + 0.00103x^2 + 0.0548x - 0.0033) - 0.0033x + 0.106} - 7.74$$

$$\hat{\theta}_0 + \hat{\theta}_1 x + \dots + \hat{\theta}_m x^m - t_{0.995,36} \sqrt{\frac{s(\hat{\theta})}{n-m-1} f^T (A^T A)^{-1} f} = -0.1505x^3 + 5.933x^2 + 4.426x - 2.719 \sqrt{x^3(0.00050x^3 - 0.00015x^2 - 0.0048x + 0.00048) + 0.00048x^3}$$

$$\sqrt{+x^2(-0.00015x^3 + 0.00210x^2 + 0.00103x - 0.0111) - 0.0111x^2 + x(-0.0048x^3 + 0.00103x^2 + 0.0548x - 0.0033) - 0.0033x + 0.106} - 7.74$$

```
# Определение векторной функции

param_vector = sp.Matrix(estimated_params)

poly_vector = sp.Matrix([1])

# Вычисление доверительных интервалов для функции

left_boundary = param_vector.T @ poly_vector - t_critical_1 * sp.sqrt((residual_sum / (total_points - degree - 1)) * poly_vector.T @ cov_matrix @ poly_vector)

right_boundary = param_vector.T @ poly_vector + t_critical_1 * sp.sqrt((residual_sum / (total_points - degree - 1)) * poly_vector.T @ cov_matrix @ poly_vector)

# Представление интервалов в NumPy

left_boundary_array = np.array(left_boundary)

right_boundary_array = np.array(right_boundary)

# Вывод доверительных интервалов

print("Доверительный интервал для phi уровня уверенности alphas = 0.95:")

print(left_boundary_array)

print("< phi(x) <")

print(right_boundary_array)

# Аналогично для уровня 0.99

left_boundary_99 = param_vector.T @ poly_vector - t_critical_2 * sp.sqrt((residual_sum / (total_points - degree - 1)) * poly_vector.T @ cov_matrix @ poly_vector)

right_boundary_99 = param_vector.T @ poly_vector + t_critical_2 * sp.sqrt((residual_sum / (total_points - degree - 1)) * poly_vector.T @ cov_matrix @ poly_vector)

left_bound_99_array = np.array(left_boundary_99)

right_bound_99_array = np.array(right_boundary_99)
```

```
# Вывод доверительных интервалов для alpha2

print("\nДоверительный интервал для phi уровня уверенности alpha2 = 0.99:")

print(left_bound_99_array)

print("< phi(x) <")

print(right_bound_99_array)
```

Доверительный интервал для phi уровня уверенности alpha1 = 0.95:

$$\begin{aligned} &[[-0.150458782848542*x^{**3} + 5.93275396662941*x^{**2} + 4.42575464386458*x - \\ &2.02809400098045*\sqrt{x^{**3}*(0.000503407166276953*x^{**3} - 0.000151022149883086*x^{**2} - 0.00481055888094256*x \\ &+ 0.000482062702426812) + 0.000482062702426812*x^{**3} + x^{**2}*(-0.000151022149883086*x^{**3} + \\ &0.00210481727751341*x^{**2} + 0.00103126553777308*x - 0.011101215375886) - 0.011101215375886*x^{**2} + \\ &x*(-0.00481055888094257*x^{**3} + 0.00103126553777308*x^{**2} + 0.0548173583437154*x - 0.00329179959657167) - \\ &0.00329179959657167*x + 0.105557297496839) - 7.7373817515195]] \end{aligned}$$

< phi(x) <

$$\begin{aligned} &[[-0.150458782848542*x^{**3} + 5.93275396662941*x^{**2} + 4.42575464386458*x + \\ &2.02809400098045*\sqrt{x^{**3}*(0.000503407166276953*x^{**3} - 0.000151022149883086*x^{**2} - 0.00481055888094256*x \\ &+ 0.000482062702426812) + 0.000482062702426812*x^{**3} + x^{**2}*(-0.000151022149883086*x^{**3} + \\ &0.00210481727751341*x^{**2} + 0.00103126553777308*x - 0.011101215375886) - 0.011101215375886*x^{**2} + \\ &x*(-0.00481055888094257*x^{**3} + 0.00103126553777308*x^{**2} + 0.0548173583437154*x - 0.00329179959657167) - \\ &0.00329179959657167*x + 0.105557297496839) - 7.7373817515195]] \end{aligned}$$

Доверительный интервал для phi уровня уверенности alpha2 = 0.99:

$$\begin{aligned} &[[-0.150458782848542*x^{**3} + 5.93275396662941*x^{**2} + 4.42575464386458*x - \\ &2.71948463044997*\sqrt{x^{**3}*(0.000503407166276953*x^{**3} - 0.000151022149883086*x^{**2} - 0.00481055888094256*x \\ &+ 0.000482062702426812) + 0.000482062702426812*x^{**3} + x^{**2}*(-0.000151022149883086*x^{**3} + \\ &0.00210481727751341*x^{**2} + 0.00103126553777308*x - 0.011101215375886) - 0.011101215375886*x^{**2} + \\ &x*(-0.00481055888094257*x^{**3} + 0.00103126553777308*x^{**2} + 0.0548173583437154*x - 0.00329179959657167) - \\ &0.00329179959657167*x + 0.105557297496839) - 7.7373817515195]] \end{aligned}$$

< phi(x) <

$$\begin{aligned} &[[-0.150458782848542*x^{**3} + 5.93275396662941*x^{**2} + 4.42575464386458*x + \\ &2.71948463044997*\sqrt{x^{**3}*(0.000503407166276953*x^{**3} - 0.000151022149883086*x^{**2} - 0.00481055888094256*x \\ &+ 0.000482062702426812) + 0.000482062702426812*x^{**3} + x^{**2}*(-0.000151022149883086*x^{**3} + \\ &0.00210481727751341*x^{**2} + 0.00103126553777308*x - 0.011101215375886) - 0.011101215375886*x^{**2} + \\ &x*(-0.00481055888094257*x^{**3} + 0.00103126553777308*x^{**2} + 0.0548173583437154*x - 0.00329179959657167) - \\ &0.00329179959657167*x + 0.105557297496839) - 7.7373817515195]] \end{aligned}$$

Задание 2.4. Графическое представление.

```
# Подсчет оценок
```

```

D_estimate = np.dot(estimated_params, poly_vector)[0]
D_function = sp.lambdify(x, D_estimate)
estimation_values = D_function(x_points)

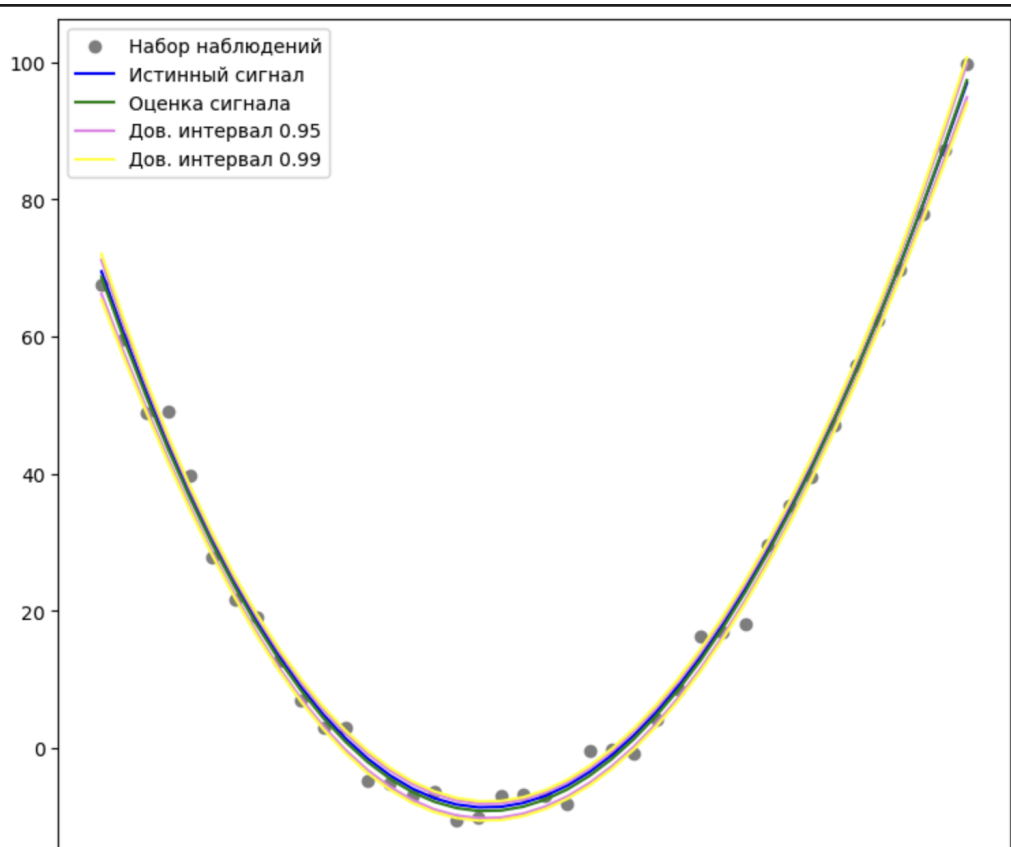
J = sp.lambdify(x, left_boundary[0])
l_conf_int = J(x_points)
J = sp.lambdify(x, right_boundary[0])
r_conf_int = J(x_points)

J = sp.lambdify(x, left_boundary_99[0])
l2_conf_int = J(x_points)
J = sp.lambdify(x, right_boundary_99[0])
r2_conf_int = J(x_points)

# Вычисление и отображение доверительных интервалов на графике
plt.figure(figsize=(9, 8))
plt.scatter(x_points, y_values, color='grey', label="Набор наблюдений")
plt.plot(x_points, phi_function, color="blue", label="Истинный сигнал")
plt.plot(x_points, estimation_values, color='green', label="Оценка сигнала")
plt.plot(x_points, l_conf_int, color='violet', label="Дов. интервал 0.95")
plt.plot(x_points, r_conf_int, color='violet')
plt.plot(x_points, l2_conf_int, color='yellow', label="Дов. интервал 0.99")
plt.plot(x_points, r2_conf_int, color='yellow')

plt.legend(loc=2)

```



Задание 2.5. Гистограмма распределения случайной ошибки.

Для построения гистограммы нужна формула Стёрджеса:

$$L+1 = 1 + [3.32 \lg n],$$

где n – количество наблюдений.

Гистограмма является оценкой плотности совместного распределения выборки, выглядит так:

$$\hat{f}_n(y) = \begin{cases} 0, & y < y_{(1)} \\ \frac{n_i}{n * \Delta}, & y \in [y_{(1)} + (i-1) * \Delta; y_{(1)} + (i) * \Delta), \\ \dots & \\ \frac{n_{L-1}}{n * \Delta_2}, & y \in [y_{(1)} + (L-2) * \Delta; y_{(n)}] \\ 0, & y > y_{(n)} \end{cases}, i = \overline{1, L-2},$$

где n_i – количество элементов, попавших в i -тый промежуток.

Для построения гистограммы желательно взять такое разбиение, расстояние между точками которого одинаково. Обозначим за $\Delta = \frac{Y_{(n)} - Y_{(1)}}{\hat{l}}$

– ширину столбца, где $Y_{(n)}$ и $Y_{(1)}$ – экстремальные статистики. Тогда

формула для $\hat{f}_n(y)$ примет следующий вид:

$$\hat{f}_n(y) = \left\{ \frac{n_i}{n\Delta}, y \in [y_{i-1}; y_i) \right\}, x \in (-\infty; y_0) \cup (y_l; +\infty), i = 1 \dots l,$$

Произведём расчеты:

$$l = 1 + [3.32 \lg n] + 2 = 6$$

$$\Delta = \frac{2.56 + 2.86}{6} \approx 0.903$$

$$\hat{f}_{40}(y) = \begin{cases} 0 & y < -2.859855560881215 \\ 0.13846923 & y \in [-2.859855560881215; -1.9571279331230613) \\ 0.08308154 & y \in [-1.9571279331230613; -1.0544003053649078) \\ 0.22155077 & y \in [-1.0544003053649078; -0.1516726776067543) \\ 0.38771385 & y \in [-0.1516726776067543; 0.7510549501513992) \\ 0.13846923 & y \in [0.7510549501513992; 1.6537825779095527) \\ 0.11077538 & y \in [1.6537825779095527; 2.5565102056677063) \\ 0 & y > 2.5565102056677063 \end{cases}$$

```

estimated_params_array = np.array(estimated_params) # Обновляем оценки параметр# Обнуление переменных перед
расчетом

y_estimation = np.zeros_like(y_values) # Инициализируем массив по размеру y_values

residuals = np.zeros_like(y_values) # Инициализируем массив остатков

# Вычисление оценки

for i in range(len(estimated_params_array)):

    y_estimation += estimated_params_array[i] * (x_points ** i)

residuals = y_values - y_estimation

max_residual = np.max(residuals)

min_residual = np.min(residuals)

# Расчет числа интервалов для гистограммы

total_points = len(y_values)

number_of_intervals = int(3.322 * math.log10(total_points)) + 1

# Расчет ширины интервала

interval_width = (max_residual - min_residual) / number_of_intervals

# Создание границ гистограммы

histogram_edges = [min_residual + interval_width * i for i in range(number_of_intervals + 1)]

# Подсчет наблюдаемых частот

observed_counts = [np.sum((residuals >= histogram_edges[i]) & (residuals < histogram_edges[i + 1]))

                    for i in range(number_of_intervals)]

observed_counts.append(0)

height = observed_counts / (total_points * interval_width)

```



```
# Построение гистограммы

plt.hist(residuals, bins=number_of_intervals, color='pink', density=True, edgecolor='grey')

print(histogram_edges)

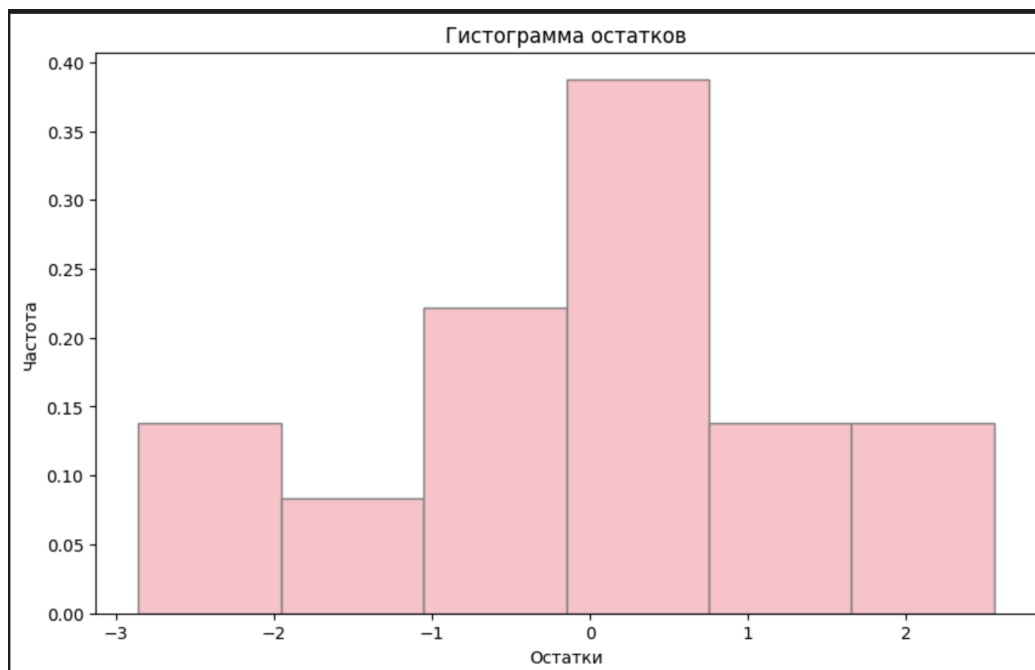
print(height)

plt.title("Гистограмма")

plt.show()
```

Границы гистограммы: [np.float64(-2.859855560881215), np.float64(-1.9571279331230613), np.float64(-1.0544003053649078), np.float64(-0.1516726776067543), np.float64(0.7510549501513992), np.float64(1.6537825779095527), np.float64(2.5565102056677063)]

Высоты гистограммы: [0.13846923 0.08308154 0.22155077 0.38771385 0.13846923 0.11077538]



Задание 2.6. Оценка дисперсии случайной ошибки.

Составим функцию правдоподобия:

$$L(E, \theta) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\theta}} e^{-\frac{E_i^2}{2\theta}} = (2\pi\theta)^{-\frac{n}{2}} e^{-\frac{1}{2\theta} \sum_{i=1}^n E_i^2};$$

$$= -\frac{n}{2} \ln \ln(2\pi) - \frac{n}{2} \ln \ln(\theta) - \frac{1}{2\theta} \sum_{i=1}^n E_i^2;$$

$$\frac{\partial \bar{L}}{\partial \theta} = -\frac{n}{2\theta} + \frac{1}{2\theta^2} \sum_{i=1}^n E_i^2 = 0;$$

$$\hat{\theta}_{\text{ММП}} = \frac{\sum_{i=1}^n E_i^2}{n} = \frac{\|E\|^2}{n} = \hat{\sigma}_{\text{ММП}}^2.$$

$$\hat{\sigma}_{\text{МП}}^2 = 1.68$$

```
variance_calculated = np.sum(residuals**2) / total_points
print("Оценка дисперсии:", variance_calculated)
```

Оценка дисперсии: 1.6818813991380028

Задание 2.7. Проверка гипотезы о том, что закон распределения ошибки наблюдения является нормальным.

Вот основная гипотеза: $H_0: E \sim N(0, \theta)$

Тогда обратная гипотеза будет иметь вид: $H_1: E \not\sim N(0, \theta)$

Статистика: $\chi^2 = np_0 + \sum_{i=1}^{\hat{l}} \frac{(n_i - np_i)^2}{np_i} + np_{\hat{l}+1},$
где

$$p_i = \Phi_0\left(\frac{x_{j+1}}{\sqrt{\hat{\sigma}_{\text{ММП}}^2}}\right) - \Phi_0\left(\frac{x_j}{\sqrt{\hat{\sigma}_{\text{ММП}}^2}}\right), -\infty = x_0 < x_1 < \dots < x_{\hat{l}} < x_{\hat{l}+1} = +\infty.$$

После расчета Z надо рассмотреть – попала ли статистика в доверительный интервал G , который имеет вид:

$$G = \left[0; k_{1-\alpha, \hat{l}-s}\right]$$

где $\hat{l} + 1$ – количество промежутков, вероятность попадания в которые не равна 0, s – количество оцениваемых параметров.

$$1 - \alpha = 1 - 0.05 = 0.95;$$

$$\hat{l} + 1 = 7 \rightarrow \hat{l} = 6; s = 1;$$

$$\sigma_{\text{ММП}}^2 = 1.68$$

$$p_0 = \Phi_0\left(y \frac{(1)}{\sqrt{\sigma_{\text{ММП}}^2}}\right) + 0.5 = \Phi_0\left(\frac{-2.86}{\sqrt{1.68}}\right) + 0.5 = \Phi_0(-2.2) + 0.5 = -0.4861 + 0.5$$

$$p_{L+1} = 0.5 - \Phi_0\left(y \frac{(n)}{\sqrt{\sigma_{\text{ММП}}^2}}\right) = 0.5 - \Phi_0\left(\frac{2.56}{\sqrt{1.68}}\right) = 0.5 - \Phi_0(1.98) = 0.5 - 0.4762$$

```
# Вычисление вероятностей

probabilities = [stats.norm.cdf(histogram_edges[0] / np.sqrt(variance_calculated))]

for i in range(number_of_intervals - 1):
    probabilities.append(stats.norm.cdf(histogram_edges[i + 1] / np.sqrt(variance_calculated)) -
stats.norm.cdf(histogram_edges[i] / np.sqrt(variance_calculated)))

probabilities.append(stats.norm.cdf(-histogram_edges[-1] / np.sqrt(variance_calculated)))

P0 = probabilities[0]

p_L_plus_1 = probabilities[-1] if len(probabilities) > 0 else 0 # Вероятность последнего интервала

print(f"Значение P0: {P0}")
print(f"Значение P(L+1): {p_L_plus_1}")

# Оценка критерия Хи-квадрат
Z_stat = total_points * probabilities[0]
for i in range(1, number_of_intervals - 1):
    Z_stat += (observed_counts[i] - total_points * probabilities[i])**2 / (total_points * probabilities[i])
Z_stat += total_points * probabilities[-1]

chi_critical_value = stats.chi2.ppf(0.95, 6)

print(chi_critical_value)
```

```
print(Z_stat)

if Z_stat <= chi_critical_value:
    print('Распределение ошибки наблюдения является нормальным')
else:
    print('Распределение ошибки наблюдения не является нормальным')
```

Значение P0: 0.013720310799307587

Значение P(L+1): 0.024345531488430435

12.591587243743977

9.008066189407266

Распределение ошибки наблюдения является нормальным

Случай 2

Вектор ε имеет равномерное распределение, то есть

$$\varepsilon_k \sim R(-3\sqrt{\sigma}, 3\sqrt{\sigma}), m = 2.$$

Сгенерированные значения: -0.90314688, -0.23010175, -1.14154911, -1.91585281, 1.80364003, 0.63553735, -0.72250782, -2.11862443, 0.61898896, 0.48333753, -1.99771932, -2.17097516, -1.28778396, -1.29625199, 1.34413977, -2.24078768, -0.71184485, -0.15105218, 1.31086854, 1.78086055, 1.36005702, -1.34280951, 1.27564142, -0.28715186, -1.38427164, 0.58910872, -1.75763866, -2.01467993, -0.30588617, -1.03276272, -1.11172918, 0.24026924, 2.27880268, 0.94259473, 0.7093318, 1.10633297, 2.1472232, -2.2177951, -2.43875827, -0.75260737

```
total_points=40
params = [param_0, param_1, param_2]

np.random.seed(420)
error_terms = np.random.uniform(-np.sqrt(3)*variance, np.sqrt(3)*variance,
total_points).T # Генерируем случайные ошибки
x_points = np.array([-4 + k * (8 /total_points) for k in
range(1,total_points+1)]).T

error_terms
```

Задание 3.1

Аналогично заданию в первом случае, получаем:

$f_1 = 4.098$	$f_2 = 4.105$	$f_3 = 4.113$
6.978	9518.04	0.6397
m=1	m=2	m=3

При m=3 статистика не попала в критическую область, значит гипотеза H_0 принимается. Получается, что порядок модели для оценки сигнала будет $\hat{m} = 3-1=2$.

МНК-оценки параметров $\hat{\theta}$ вычисляются по формуле:

$$\hat{\theta} = (A^T A)^{-1} A^T Y$$

```
def estimate_parameters(degree, transposed_matrix, observed_y):

    mat_a = transposed_matrix.T

    cov_matrix = np.linalg.inv(np.dot(transposed_matrix, mat_a))

    temp_vector = np.dot(transposed_matrix, observed_y)

    temp_vector = temp_vector.T

    estimated_params = np.dot(cov_matrix, temp_vector)

    variance_jj = cov_matrix[degree, degree]

    errors = observed_y - np.dot(mat_a, estimated_params)

    sum_squared_errors = np.dot(errors.T, errors)

    Z_statistic = (total_points - (degree + 1)) * (estimated_params[degree] ** 2) / (variance_jj *
sum_squared_errors)

    return Z_statistic, estimated_params, sum_squared_errors, cov_matrix

m = 1

mat_a1T = np.array([np.ones(40), x_points])

Z_statistic, estimated_params, sum_squared_errors, cov_matrix = estimate_parameters(m, mat_a1T, y_values)
```

```

print("m = 1 :", Z_statistic)

print(estimated_params)

m = 2

mat_a2T = np.array([np.ones(40), x_points, x_points**2])

Z_statistic, estimated_params, sum_squared_errors, cov_matrix = estimate_parameters(m, mat_a2T, y_values)

print("m = 2 :", Z_statistic)

print(estimated_params)

m = 3

mat_a3T = np.array([np.ones(40), x_points, x_points**2, x_points**3 ])

Z_statistic, estimated_params, sum_squared_errors, cov_matrix = estimate_parameters(m, mat_a3T, y_values)

print("m = 3 :", Z_statistic)

print(estimated_params)

```

Степень 1: Z = 6.919931248338522, Параметры: [23.59036741 5.26943746]

Степень 2: Z = 16024.175009761757, Параметры: [-8.21608034 4.07370634 5.97865559]

Степень 3: Z = 0.6397084790468818, Параметры: [-8.23405221 4.25304975 5.98428588 -0.01876762]

Степень 4: Z = 0.523267154057975, Параметры: [-8.41865946 4.2297797 6.10029535 -0.01535988
-0.00851937]

$$\hat{\theta}_{(1 \times 3)} = (-8.21608034 \ 4.07370634 \ 5.978655593)$$

Задание 3.2. Доверительные интервалы для параметров.

Задание тоже аналогично первому случаю, для построения доверительных интервалов уровня надежности $\alpha_1 = 0.95$ и $\alpha_2 = 0.99$ для параметров модели, определенных в предыдущем пункте, будем использовать следующую формулу:

$$P\left(\hat{\theta}_i - t_{1-\frac{\alpha}{2}, n-m-1} \sqrt{\hat{\sigma}^2 c_{ii}} \leq \theta_i \leq \hat{\theta}_i + t_{1-\frac{\alpha}{2}, n-m-1} \sqrt{\hat{\sigma}^2 c_{ii}}\right) = 1 - \alpha,$$

где $\hat{\sigma}^2 = \frac{s(\hat{\theta})}{n-m-1} = \frac{\|E\|^2}{n-m-1}$ – оценка дисперсии ошибок, $t_{1-\frac{\alpha}{2}, n-m-1}$ – экстремальная статистика для уровня надежности $[\alpha = 1 - \alpha_1 \quad \alpha = 1 - \alpha_2, m - \text{порядок модели}, c_{ii} - \text{элемент на пересечении } i \text{ столбца и } i \text{ строки матрицы } (A^T A)^{-1}]$.

$$t_{0.975,36} = 2.0281; t_{0.995,36} = 2.7195$$

$$s(\hat{\theta}) = (Y - A\hat{\theta})^T (Y - A\hat{\theta}) = 74.89$$

$$\hat{\theta}_0 - t_{0.975,36} \sqrt{\frac{c_{11}s(\hat{\theta})}{40-4}} = -8.900$$

$$\hat{\theta}_0 + t_{0.975,36} \sqrt{\frac{c_{11}s(\hat{\theta})}{40-4}} = -7.532$$

$$\hat{\theta}_0 - t_{0.995,36} \sqrt{\frac{c_{11}s(\hat{\theta})}{40-4}} = -9.132$$

$$\hat{\theta}_0 + t_{0.995,36} \sqrt{\frac{c_{11}s(\hat{\theta})}{40-4}} = -7.300$$

```
# Определение границ доверительных интервалов

degrees_of_freedom = total_points - (degree_res + 1)

confidence_level_1 = 0.95

confidence_level_2 = 0.99

t_critical_1 = scipy.stats.t.ppf(0.975, degrees_of_freedom)

t_critical_2 = scipy.stats.t.ppf(0.995, degrees_of_freedom)

print("Критические значения t для уровней уверенности:")

print(t_critical_1)

print(t_critical_2)
```

```

# Сообщение о доверительных интервалах

print('\nДля уровня надежности alpha1 = 0.95:')

for i in range(degree_res + 1):

    conf_interval_left = estimated_params[i] - t_critical_1 *
math.sqrt(cov_matrix[i][i] * sum_squared_errors / (total_points - degree - 1))

    conf_interval_right = estimated_params[i] + t_critical_1 *
math.sqrt(cov_matrix[i][i] * sum_squared_errors / (total_points - degree - 1))

    print(f'Для параметра {i}: {conf_interval_left} <= theta{i} <=
{conf_interval_right}')

print('\nДля уровня надежности alpha2 = 0.99:')

for i in range(degree_res + 1):

    conf_interval_left = estimated_params[i] - t_critical_2 *
math.sqrt(cov_matrix[i][i] * sum_squared_errors / (total_points - degree - 1))

    conf_interval_right = estimated_params[i] + t_critical_2 *
math.sqrt(cov_matrix[i][i] * sum_squared_errors / (total_points - degree - 1))

    print(f'Для параметра {i}: {conf_interval_left} <= theta{i} <=
{conf_interval_right}')

```

Для уровня надежности alpha1 = 0.95:

Доверительный интервал для параметра 0: -8.89968779520921 <= theta0 <= -7.532472876576272

Доверительный интервал для параметра 1: 3.8753580221073056 <= theta1 <= 4.272054663699021

Доверительный интервал для параметра 2: 5.882959016491197 <= theta2 <= 6.074352165259082

Для уровня надежности $\alpha_2 = 0.99$:

Доверительный интервал для параметра 0: $-9.13221919782961 \leq \theta_0 \leq -7.2999414739558715$

Доверительный интервал для параметра 1: $3.807889166094267 \leq \theta_1 \leq 4.33952351971206$

Доверительный интервал для параметра 2: $5.850407501196564 \leq \theta_2 \leq 6.106903680553716$

Задание 3.3. Доверительные интервалы для полезного сигнала.

Для построения доверительных интервалов уровня надежности $\alpha_1 = 0.95$ и $\alpha_2 = 0.99$ для полезного сигнала, будем использовать следующую формулу:

$$P\left(\sum_{i=0}^m \hat{\theta}_i x^i - t_{1-\frac{\alpha}{2}, n-m-1} \sqrt{X^T (A^T A)^{-1} X \frac{s(\hat{\theta})}{n-m-1}} \leq \sum_{i=0}^m \theta_i x^i \leq \sum_{i=0}^m \hat{\theta}_i x^i + t_{1-\frac{\alpha}{2}, n-m-1} \sqrt{X^T (A^T A)^{-1} X \frac{s(\hat{\theta})}{n-m-1}}\right)$$

где $X = (1 \ x \ : \ x^m)$, $\frac{s(\hat{\theta})}{n-m-1} = \frac{\|E\|^2}{n-m-1}$ – оценка дисперсии ошибок, $t_{1-\frac{\alpha}{2}, n-m-1}$ – экстремальная статистика для уровня надежности $[\alpha = 1 - \alpha_1 \ \alpha = 1 - \alpha_2]$, m – порядок модели, вычисленный в задании 2.1

$$\hat{\theta}_0 + \hat{\theta}_1 x + \dots + \hat{\theta}_m x^m - t_{0.975, 36} \sqrt{\frac{s(\hat{\theta})}{n-m-1} f^T (A^T A)^{-1} f} = 5.98x^2 + 4.07x - 2.026 \sqrt{x^2(0.00223x^2 - 0.000446 - 0.011)}$$

$$\sqrt{-0.012 + x(-0.000446x^2 + 0.0096x + 0.0014x) - 0.0014x + 0.114} - 8.22$$

$$\hat{\theta}_0 + \hat{\theta}_1 x + \dots + \hat{\theta}_m x^m + t_{0.975, 36} \sqrt{\frac{s(\hat{\theta})}{n-m-1} f^T (A^T A)^{-1} f} = 5.98x^2 + 4.07x - 2.026 \sqrt{x^2(0.00223x^2 - 0.000446 - 0.011)}$$

$$\sqrt{-0.012 + x(-0.000446x^2 + 0.0096x + 0.0014x) - 0.0014x + 0.114} + 8.22$$

```
# Определение векторной функции
param_vector = sp.Matrix(estimated_params)
x = sp.Symbol('x')
poly_vector = sp.Matrix([1, x, x**2])

left_boundary = param_vector.T @ poly_vector - t_critical_1 * sp.sqrt((sum_squared_errors / degrees_of_freedom)
* poly_vector.T @ cov_matrix @ poly_vector)
right_boundary = param_vector.T @ poly_vector + t_critical_1 * sp.sqrt((sum_squared_errors /
degrees_of_freedom) * poly_vector.T @ cov_matrix @ poly_vector)

# Представление интервалов в NumPy
left_boundary_array = np.array(left_boundary)
```

```

right_boundary_array = np.array(right_boundary)

# Вывод доверительных интервалов
print("Доверительный интервал для phi уровня уверенности alpha1 = 0.95:")
print(left_boundary_array)
print("< phi(x) <")
print(right_boundary_array)

# Аналогично для уровня 0.99
left_boundary_99 = param_vector.T @ poly_vector - t_critical_2 * sp.sqrt((sum_squared_errors /
degrees_of_freedom) * poly_vector.T @ cov_matrix @ poly_vector)
right_boundary_99 = param_vector.T @ poly_vector + t_critical_2 * sp.sqrt((sum_squared_errors /
degrees_of_freedom) * poly_vector.T @ cov_matrix @ poly_vector)

left_bound_99_array = np.array(left_boundary_99)
right_bound_99_array = np.array(right_boundary_99)

# Вывод доверительных интервалов для alpha2
print("\nДоверительный интервал для phi уровня уверенности alpha2 = 0.99:")
print(left_bound_99_array)
print("< phi(x) <")
print(right_bound_99_array)

```

Доверительный интервал для phi уровня уверенности alpha1 = 0.95:

$$\begin{aligned}
 & [[5.97865559087514*x**2 + 4.07370634290316*x - \\
 & 2.02619246302911*sqrt(x**2*(0.00223064979336081*x**2 - 0.000446129958672163*x - \\
 & 0.0118670569006795) - 0.0118670569006795*x**2 + x*(-0.000446129958672163*x**2 + \\
 & 0.00958287151227805*x + 0.00142404682808154) + 0.00142404682808154*x + 0.113828809791318) - \\
 & 8.21608033589274]] < \phi(x) < [[5.97865559087514*x**2 + 4.07370634290316*x + \\
 & 2.02619246302911*sqrt(x**2*(0.00223064979336081*x**2 - 0.000446129958672163*x - \\
 & 0.0118670569006795) - 0.0118670569006795*x**2 + x*(-0.000446129958672163*x**2 + \\
 & 0.00958287151227805*x + 0.00142404682808154) + 0.00142404682808154*x + 0.113828809791318) - \\
 & 8.21608033589274]]
 \end{aligned}$$

Доверительный интервал для phi уровня уверенности alpha2 = 0.99:

$$\begin{aligned}
 & [[5.97865559087514*x**2 + 4.07370634290316*x - \\
 & 2.71540872154996*sqrt(x**2*(0.00223064979336081*x**2 - 0.000446129958672163*x - \\
 & 0.0118670569006795) - 0.0118670569006795*x**2 + x*(-0.000446129958672163*x**2 + \\
 & 0.00958287151227805*x + 0.00142404682808154) + 0.00142404682808154*x + 0.113828809791318) - \\
 & 8.21608033589274]] < \phi(x) < [[5.97865559087514*x**2 + 4.07370634290316*x + \\
 & 2.71540872154996*sqrt(x**2*(0.00223064979336081*x**2 - 0.000446129958672163*x - \\
 & 0.0118670569006795) - 0.0118670569006795*x**2 + x*(-0.000446129958672163*x**2 + \\
 & 0.00958287151227805*x + 0.00142404682808154) + 0.00142404682808154*x + 0.113828809791318) - \\
 & 8.21608033589274]]
 \end{aligned}$$

Задание 3.4. Графическое представление.

```

# Подсчет оценок

D_estimate = np.dot(estimated_params, poly_vector)[0]

```

```

D_function = sp.lambdify(x, D_estimate)

estimation_values = D_function(x_points)

J = sp.lambdify(x, left_boundary[0])

l_conf_int = F(x_points)

J = sp.lambdify(x, right_boundary[0])

r_conf_int = F(x_points)

J = sp.lambdify(x, left_boundary_99[0])

l2_conf_int = F(x_points)

J = sp.lambdify(x, right_boundary_99[0])

r2_conf_int = F(x_points)

# Вычисление и отображение доверительных интервалов на графике

plt.figure(figsize=(9, 8))

plt.scatter(x_points, y_values, color='grey', label="Набор наблюдений")

plt.plot(x_points, phi_function, color="blue", label="Истинный сигнал")

plt.plot(x_points, estimation_values, color='green', label="Оценка сигнала")

plt.plot(x_points, l_conf_int, color='violet', label="Дов. интервал 0.95")

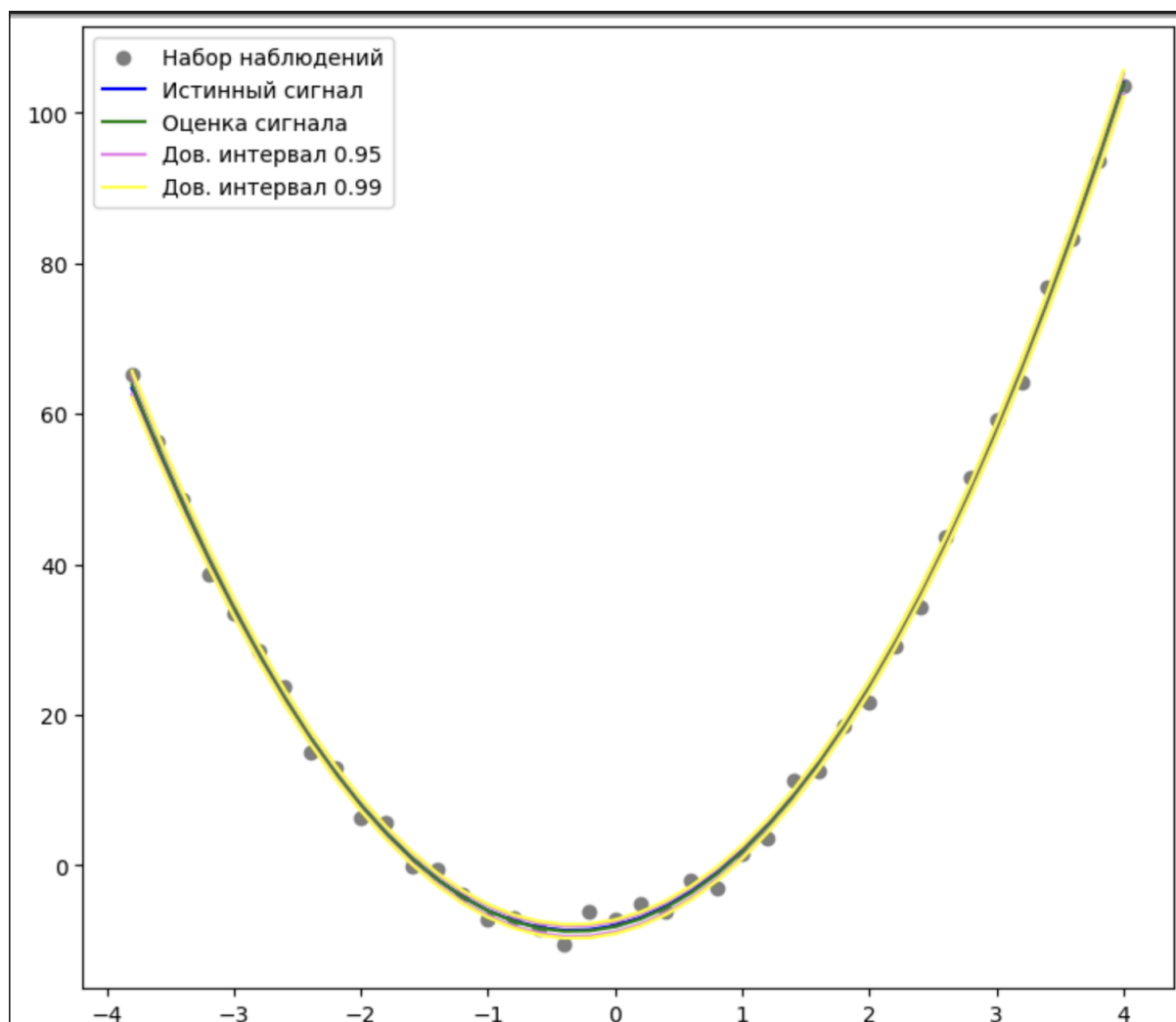
plt.plot(x_points, r_conf_int, color='violet')

plt.plot(x_points, l2_conf_int, color='yellow', label="Дов. интервал 0.99")

plt.plot(x_points, r2_conf_int, color='yellow')

plt.legend(loc=2)

```



Задание 3.5. Гистограмма распределения случайной ошибки.

Для построения гистограммы нужна формула Стёрджеса:

$$l+1 = 1 + [3.32 \lg n],$$

где n – количество наблюдений.

Гистограмма является оценкой плотности совместного распределения выборки, выглядит так:

$$\hat{f}_n(x) = \left\{ \frac{n_i}{n(x_i - x_{i-1})}, x \in [x_{i-1}; x_i) \right\}, 0, x \in (-\infty; x_0) \cup (x_l; +\infty), i = 1 \dots l,$$

где n_i – количество элементов, попавших в i -тый промежуток.

Формула для $\hat{f}_n(x)$ примет следующий вид:

$$\hat{f}_n(y) = \begin{cases} 0, & y < y_{(1)} \\ \frac{n_i}{n * \Delta}, & y \in [y_{(1)} + (i - 1) * \Delta; y_{(1)} + (i) * \Delta), \\ \dots & \\ \frac{n_{L-1}}{n * \Delta_2}, & y \in [y_{(1)} + (L - 2) * \Delta; y_{(n)}] \\ 0, & y > y_{(n)} \end{cases}, i = \overline{1, L - 2},$$

$$\hat{f}_{40}(y) = \begin{cases} 0 & y < -2.859855560881215 \\ 0.2585046 & y \in [-2.859855560881215; -1.4208681872473945) \\ 0.22619154 & y \in [-1.4208681872473945; -0.6471876041271567) \\ 0.25850461 & y \in [-0.6471876041271567; 0.1264929789930811) \\ 0.12925231 & y \in [0.1264929789930811; 0.9001735621133187) \\ 0.29081769 & y \in [0.9001735621133187; 1.6738541452335562) \\ 0.09693923 & y \in [1.6738541452335562; 2.4475347283537943) \\ 0 & y > 2.4475347283537943 \end{cases}$$

Произведём расчеты:

$$l = [3.322 \lg n] + 1 + 2 = 6$$

$$\Delta = \frac{2.44 + 2.19}{6} \approx 0,772$$

$$\hat{f}_n(x) = \{0, x \in (-\infty; -2.19) 0.259, x \in [-2.19; -1.42) 0.226, x \in [-1.42; -0.65)$$

```
estimated_params_array = np.array(estimated_params)
y_estimation = sum([estimated_params_array[i] * (x_points ** i) for i in
range(len(estimated_params_array))])
residuals = y_values - y_estimation
max_residual = max(residuals)
min_residual = min(residuals)

number_of_intervals = int(3.322 * math.log10(total_points)) + 1
interval_width = (max_residual - min_residual) / number_of_intervals
```

```

histogram_edges = [min_residual + interval_width * i for i in
range(number_of_intervals + 1)]

observed_counts = [np.sum((residuals >= histogram_edges[i]) & (residuals <
histogram_edges[i + 1])) for i in range(number_of_intervals)]
observed_counts.append(0)

plt.hist(residuals, bins=number_of_intervals, color='pink', density=True,
edgecolor='grey')

height = observed_counts / (total_points * interval_width)

# Построение гистограммы
plt.hist(residuals, bins=number_of_intervals, color='pink', density=True,
edgecolor='grey')

print(histogram_edges)

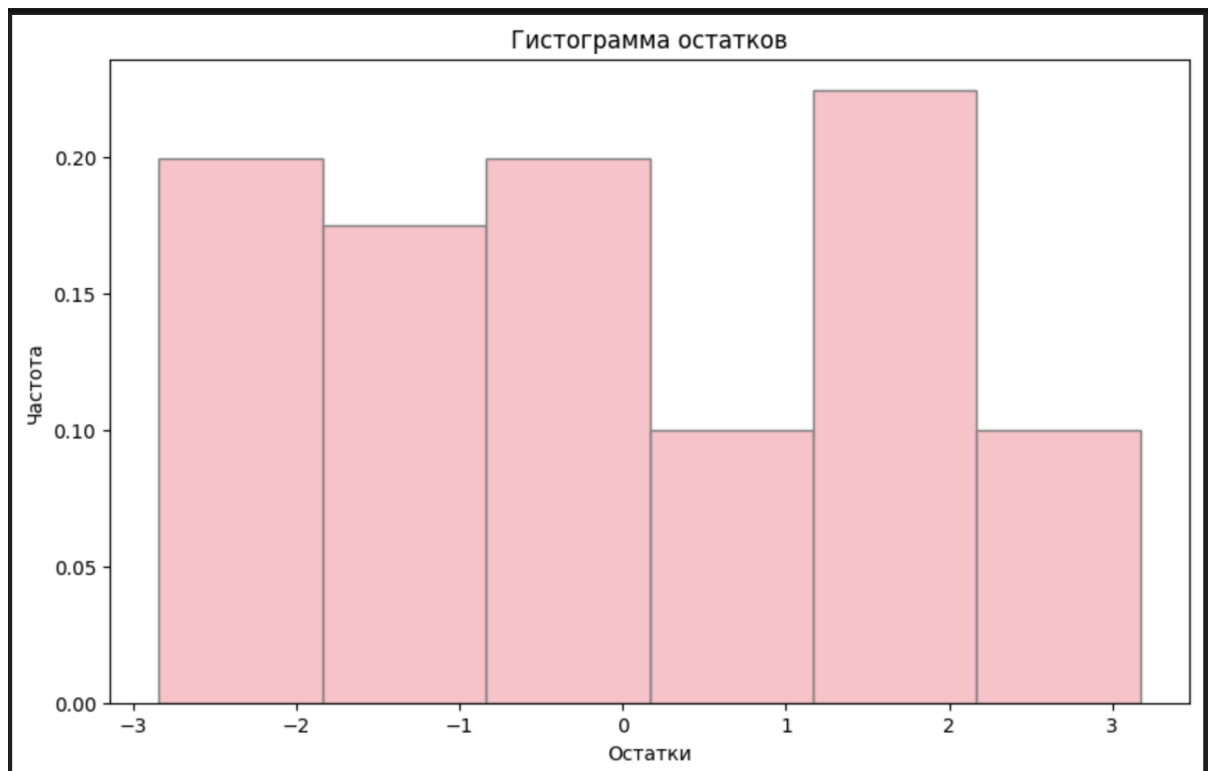
print(height)

plt.title("Гистограмма")
plt.show()

```

Границы гистограммы: [np.float64(-2.194548770367632), np.float64(-1.4208681872473945),
np.float64(-0.6471876041271567), np.float64(0.1264929789930811),
np.float64(0.9001735621133187), np.float64(1.6738541452335562),
np.float64(2.4475347283537943)]

Высоты гистограммы: [0.25850461 0.22619154 0.25850461 0.12925231 0.29081769 0.09693923]



Задание 3.6. Оценка дисперсии случайной ошибки.

Аналогично первому случаю проводим действия.

Составим функцию правдоподобия:

$$L(E, \theta) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\theta}} e^{-\frac{E_i^2}{2\theta}} = (2\pi\theta)^{-\frac{n}{2}} e^{-\frac{1}{2\theta} \sum_{i=1}^n E_i^2};$$

$$= -\frac{n}{2} \ln(2\pi) - \frac{n}{2} \ln(\theta) - \frac{1}{2\theta} \sum_{i=1}^n E_i^2;$$

$$\frac{\partial L}{\partial \theta} = -\frac{n}{2\theta} + \frac{1}{2\theta^2} \sum_{i=1}^n E_i^2 = 0;$$

$$\hat{\theta}_{\text{ММП}} = \frac{\sum_{i=1}^n E_i^2}{n} = \frac{\|E\|^2}{n} = \hat{\sigma}_{\text{ММП}}^2.$$

$$\hat{\sigma}_{\text{МП}}^2 = 1.87$$

```
variance_calculated = np.sum(residuals**2) / total_points
print("Оценка дисперсии:", variance_calculated)
```

Оценка дисперсии: 1.8722418331064072

Задание 3.7. Проверка гипотезы о том, что закон распределения ошибки наблюдения является нормальным.

Вот основная гипотеза: $H_0: E \sim N(0, \theta)$

Тогда обратная гипотеза будет иметь вид: $H_1: E \not\sim N(0, \theta)$

$$\text{Статистика: } \chi^2 = np_0 + \sum_{i=1}^{\hat{l}} \frac{(n_i - np_i)^2}{np_i} + np_{\hat{l}+1},$$

где

$$p_i = \Phi_0\left(\frac{x_{j+1}}{\sqrt{\hat{\sigma}_{\text{ММП}}^2}}\right) - \Phi_0\left(\frac{x_j}{\sqrt{\hat{\sigma}_{\text{ММП}}^2}}\right), -\infty = x_0 < x_1 < \dots < x_{\hat{l}} < x_{\hat{l}+1} = +\infty.$$

После расчета Z надо рассмотреть – попала ли статистика в

доверительный интервал G , который имеет вид:

$$G = \left[0; k_{1-\alpha, \hat{l}-s}\right]$$

где $\hat{l} + 1$ – количество промежутков, вероятность попадания в которые не равна 0, s – количество оцениваемых параметров.

$$1 - \alpha = 1 - 0.05 = 0.95;$$

$$\hat{l} + 1 = 7 \rightarrow \hat{l} = 6; s = 1;$$

$$\hat{\sigma}_{\text{ММП}}^2 = 1.87$$

$$p_0 = \Phi_0\left(y \frac{(1)}{\sqrt{\hat{\sigma}_{\text{ММП}}^2}}\right) + 0.5 = \Phi_0\left(\frac{-2.19}{\sqrt{1.87}}\right) + 0.5 = \Phi_0(-1.60) + 0.5 = 0.4452 + 0,$$

$$p_{L+1} = 0.5 - \Phi_0\left(y \frac{(n)}{\sqrt{\hat{\sigma}_{\text{ММП}}^2}}\right) = 0.5 - \Phi_0\left(\frac{2.44}{\sqrt{1.87}}\right) = 0.5 - \Phi_0(1.79) = 0.5 - 0.4633$$


```

# Вычисление вероятностей

probabilities = [stats.norm.cdf(histogram_edges[0] / np.sqrt(variance_calculated))]

for i in range(number_of_intervals - 1):
    probabilities.append(stats.norm.cdf(histogram_edges[i + 1] / np.sqrt(variance_calculated)) -
stats.norm.cdf(histogram_edges[i] / np.sqrt(variance_calculated)))

probabilities.append(stats.norm.cdf(-histogram_edges[-1] / np.sqrt(variance_calculated)))
print(histogram_edges[0] / np.sqrt(variance_calculated))
print(stats.norm.cdf(histogram_edges[0] / np.sqrt(variance_calculated))-0.5)
print(probabilities)

# Оценка критерия Хи-квадрат
Z_stat = total_points * probabilities[0]
for i in range(1, number_of_intervals - 1):
    Z_stat += (observed_counts[i] - total_points * probabilities[i])**2 / (total_points * probabilities[i])
Z_stat += total_points * probabilities[-1]

chi_critical_value = stats.chi2.ppf(0.95, 6)

print(chi_critical_value)
print(Z_stat)

if Z_stat <= chi_critical_value:
    print('Распределение ошибки наблюдения является нормальным')
else:
    print('Распределение ошибки наблюдения не является нормальным')

```

Значение P0: 0.0453054597782205

Значение P(L+1): 0.029562844233171015

12.591587243743977

12.603724034307387

Закон распределения ошибки наблюдения не является нормальным

Вывод:

В ходе выполнения своей курсовой работы я исследовала метод наименьших квадратов (МНК), который применялся для анализа данных с нормальным и равномерным распределением ошибок. Полученные результаты подтвердили эффективность этого подхода при различных законах распределения погрешностей.

Я также изучила, как выбирать порядок многочлена, опираясь на критерий Фишера, и оценила гипотезу о нормальности распределения ошибок с помощью критерия Пирсона.

Визуализация данных была представлена в виде гистограмм, что позволило мне наглядно оценить плотности распределений. Кроме того, я построила доверительные интервалы как для оценок параметров, так и для полезного сигнала, что добавило глубины моему анализу.